AI-Powered CAD Comparison Tool

Final Walkthrough & Project Summary

Congratulations! We have successfully completed all 11 phases of the AI-powered CAD Comparison Engine. The final application seamlessly blends traditional computer vision with modern deep learning to analyze, highlight, and semantically summarize the differences between engineering drawings.

Architecture & Pipeline

The system processes incoming drawing pairs through a robust 6-step pipeline:

1. **Input & Alignment (`input_handler.py`)**: Converts PDF/PNG/JPG inputs to high-resolution OpenCV arrays. Uses **ORB feature matching** and RANSAC homography to mathematically align the "After" drawing to the "Before" drawing, correcting minor rotations and translations.
2. **Pixel-Level Difference Engine (`pixel_diff.py`)**: Computes structural similarity (**SSIM**) and absolute intensity differences. It handles extremely large images (auto-downscaling images >20M pixels for SSIM to prevent OOM errors) and outputs a localized heatmap and binary masks.
3. **Semantic Object Detection (`yolo_detector.py`)**: Runs our custom-trained **YOLOv8** model (trained on your Roboflow dataset) to locate `dimension` annotations and `Notes` across both drawings.
4. **Optical Character Recognition (`ocr_engine.py`)**: Uses **PaddleOCR** (running in CPU mode for stability) to extract the actual text values from the regions identified by YOLO.
5. **Element Matching Engine (`element_matcher.py`)**: The core AI logic. It uses the **Hungarian Algorithm** to mathematically pair the OCR-enriched detections from both images. By evaluating spatial distance (Euclidean center distance), bounding box overlap (IoU), and text differences, it accurately classifies changes into:
   - `Added` (green)
   - `Removed` (red)
   - `Modified` (yellow)
   - `Moved` (cyan)
6. **Statistics & AI Summary (`statistics.py`, `summarizer.py`)**: Calculates the percentage of pixels changed and isolates **Unlabeled Geometry Changes** (structural changes that lack text annotations). Finally, it dynamically guesses the drawing type (e.g., "Bridge / Infrastructure") and outputs a clean, readable Markdown report.

The Streamlit Interface

The entire backend pipeline is wrapped in a modern, interactive web interface via `app.py`.

- **Model Caching**: YOLO and OCR models are cached in RAM via `@st.cache_resource` so they only load once, ensuring subsequent drawing comparisons are snappy.
- **Progress Tracking**: A dynamic progress bar updates the user through all 6 heavy computation steps.
- **Interactive Toggles**: Users can seamlessly toggle the visualizer between the `Combined Overlay` (Heatmap + Semantic Boxes), the `Original Before/After` drawings, or the `Pure Heatmap`.
- **Report Generation**: The AI summary report is rendered directly in the app, with a convenient download button for exporting it to Markdown format.

Final Deliverables

You will find the following core scripts in your `app/` directory: - `__init__.py` - `input_handler.py` - `pixel_diff.py` - `yolo_detector.py` - `ocr_engine.py` - `element_matcher.py` - `statistics.py` - `summarizer.py` - `visualizer.py`

And the main entry point in the root: - `app.py`

Note: The best-performing YOLOv8 weights are stored in `models/best.pt`.

How to Run

To launch the web interface locally, ensure your virtual environment is activated and run:

```
source venv/bin/activate
streamlit run app.py
```